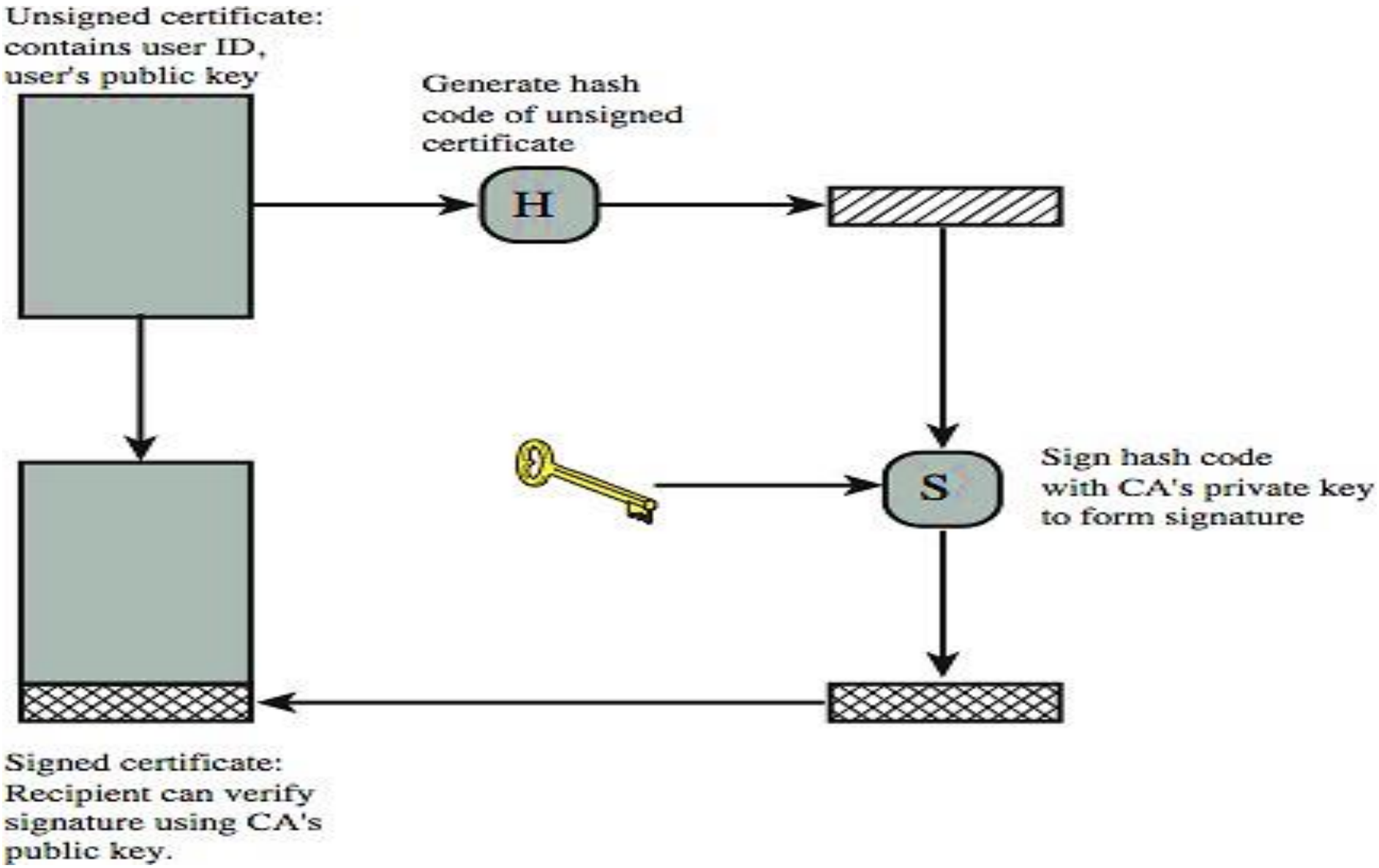
X.509 Certificates

Authentication Service

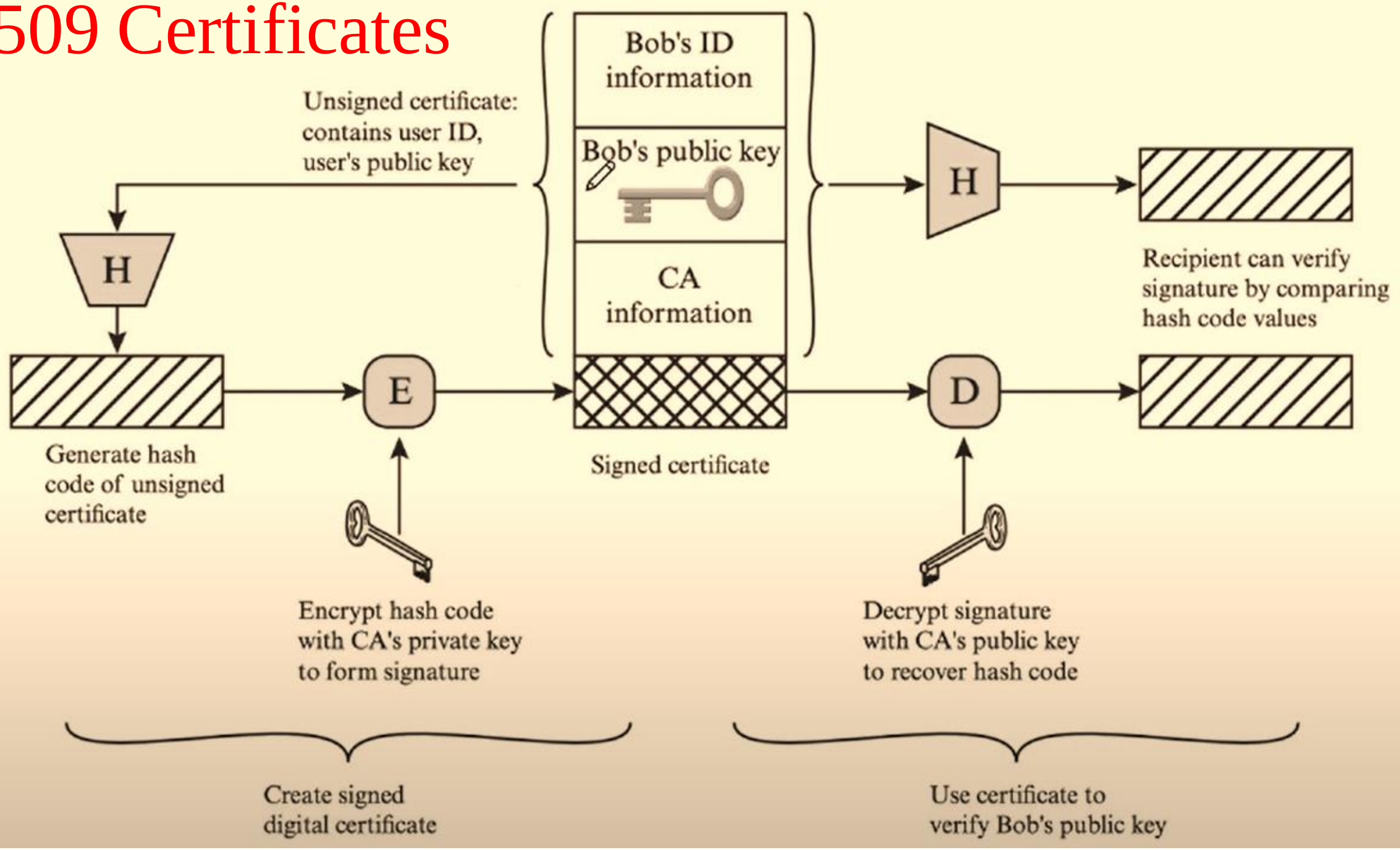
X.509 Authentication Service

- part of CCITT X.500 directory service standards
  - distributed servers maintaining user info database
    - **defines framework for authentication services**
      - directory may store public-key certificates
  - with public key of user signed by certification authority
- also defines authentication protocols
- **uses public-key crypto & digital signatures**
  - algorithms not standardised, but RSA recommended
- X.509 certificates are widely used
  - **have 3 versions**

X.509 Certificates



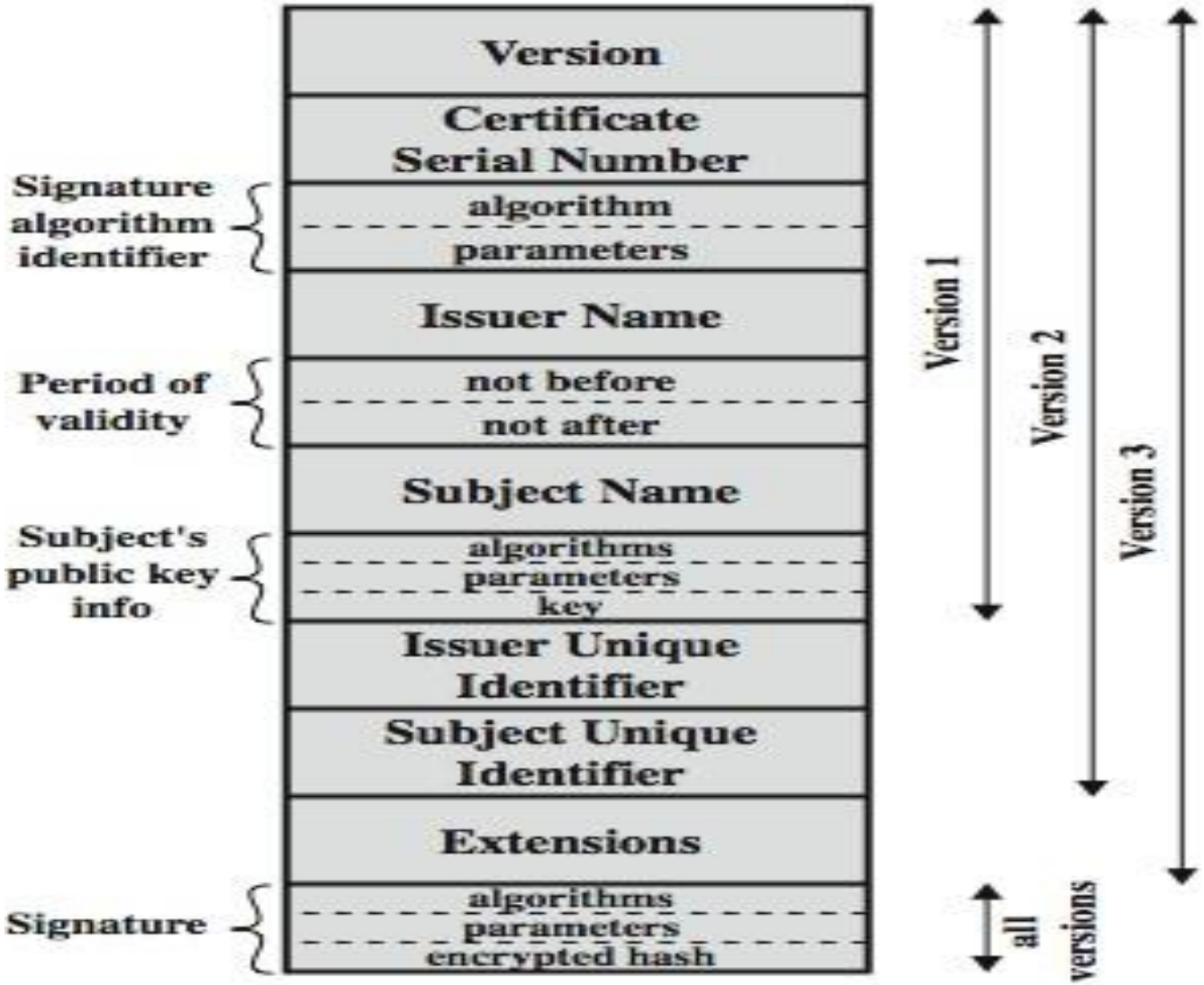
X.509 Certificates



X.509 Certificates … Contd.

- issued by a Certification Authority (CA), containing:
  - version V (1, 2, or 3)
  - serial number SN (*unique within CA*) identifying certificate
  - signature algorithm identifier AI
  - issuer (*X.509 name CA*)
  - period of validity TA (*from - to dates*)
  - subject X.509 name A (*name of owner*)
  - subject public-key info Ap (*algorithm, parameters, key*)
  - issuer unique identifier (v2+)
  - subject unique identifier (v2+)
  - extension fields (v3)
  - signature (of hash of all fields in certificate)
- **notation CA<<A>> denotes certificate for A signed by CA**

X.509 Certificates … Contd.



(a) X.509 Certificate

The standard uses the following notation to define a certificate:

$$CA \ll A \gg = CA\{V, SN, AI, CA, UCA, A, UA, Ap, T^A\}$$

where

$Y \ll X \gg$ = the certificate of user X issued by certification authority Y

$Y\{I\}$ = the signing of I by Y. It consists of I with an encrypted has code appended

V = version of the certificate

SN = serial number of the certificate

AI = identifier of the algorithm used to sign the certificate

CA = name of certificate authority

UCA = optional unique identifier of the CA

A = name of user A

UA = optional unique identifier of the user A

Ap = public key of user A

T^A = period of validity of the certificate

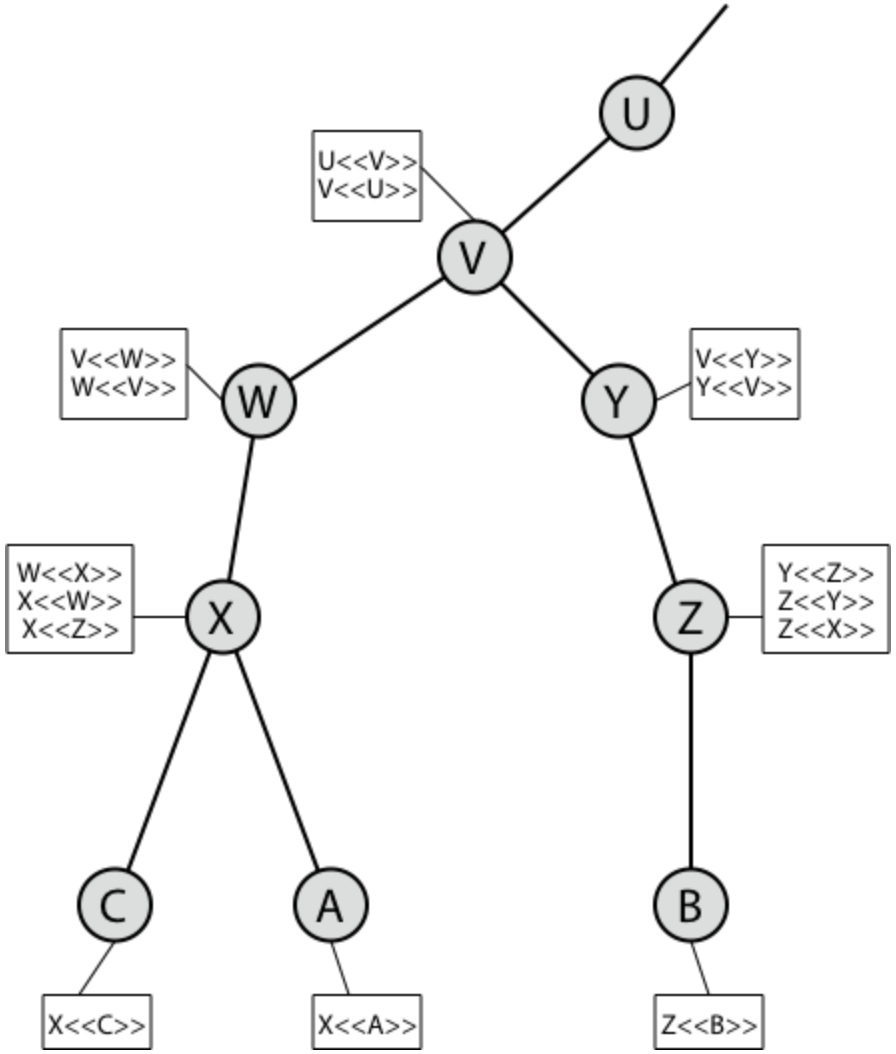
Obtaining a Certificate

- any user with access to CA can get any certificate from it
- only the CA can modify a certificate
- because cannot be forged, certificates can be placed in a public directory

CA Hierarchy

- if both users share a common CA then they are assumed to know its public key
- otherwise CA's must form a hierarchy
- use certificates linking members of hierarchy to validate other CA's
  - each CA has certificates for clients (forward) and parent (backward)
- each client trusts parents certificates
- enable verification of any certificate from one CA by users of all other CAs in hierarchy

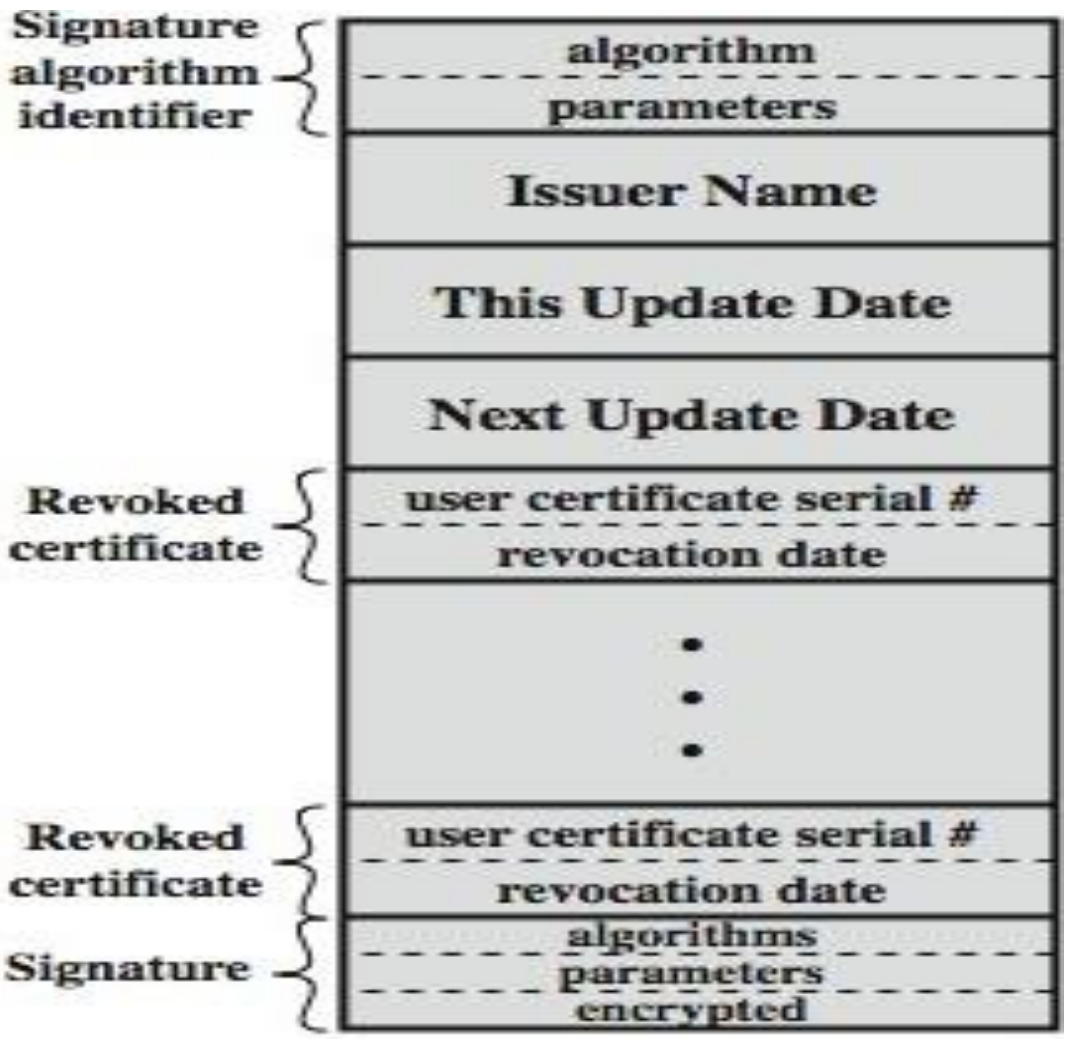
Obtaining a Certificate … Contd.



Certificate Revocation

- certificates have a period of validity
- may need to revoke before expiry, eg:
  - user's private key is compromised
  - user is no longer certified by this CA
  - CA's certificate is compromised
- CA’s maintain list of revoked certificates
  - the Certificate Revocation List (CRL)
- users should check certificates with CA’s CRL

X.509 Certificates … Contd.



(b) Certificate Revocation List

X.509 V3(Version 3)

- has been recognised that additional information is needed in a certificate
  - email/URL, policy details, usage constraints
- rather than explicitly naming new fields defined a general extension method
- extensions consist of:
  - extension identifier
  - criticality indicator
  - extension value

Certificate Extensions

- key and policy information
  - convey info about subject & issuer keys, plus indicators of certificate policy
- certificate subject and issuer attributes
  - support alternative names, in alternative formats for certificate subject and/or issuer
- certificate path constraints
  - allow constraints on use of certificates by other CA's

Kerberos

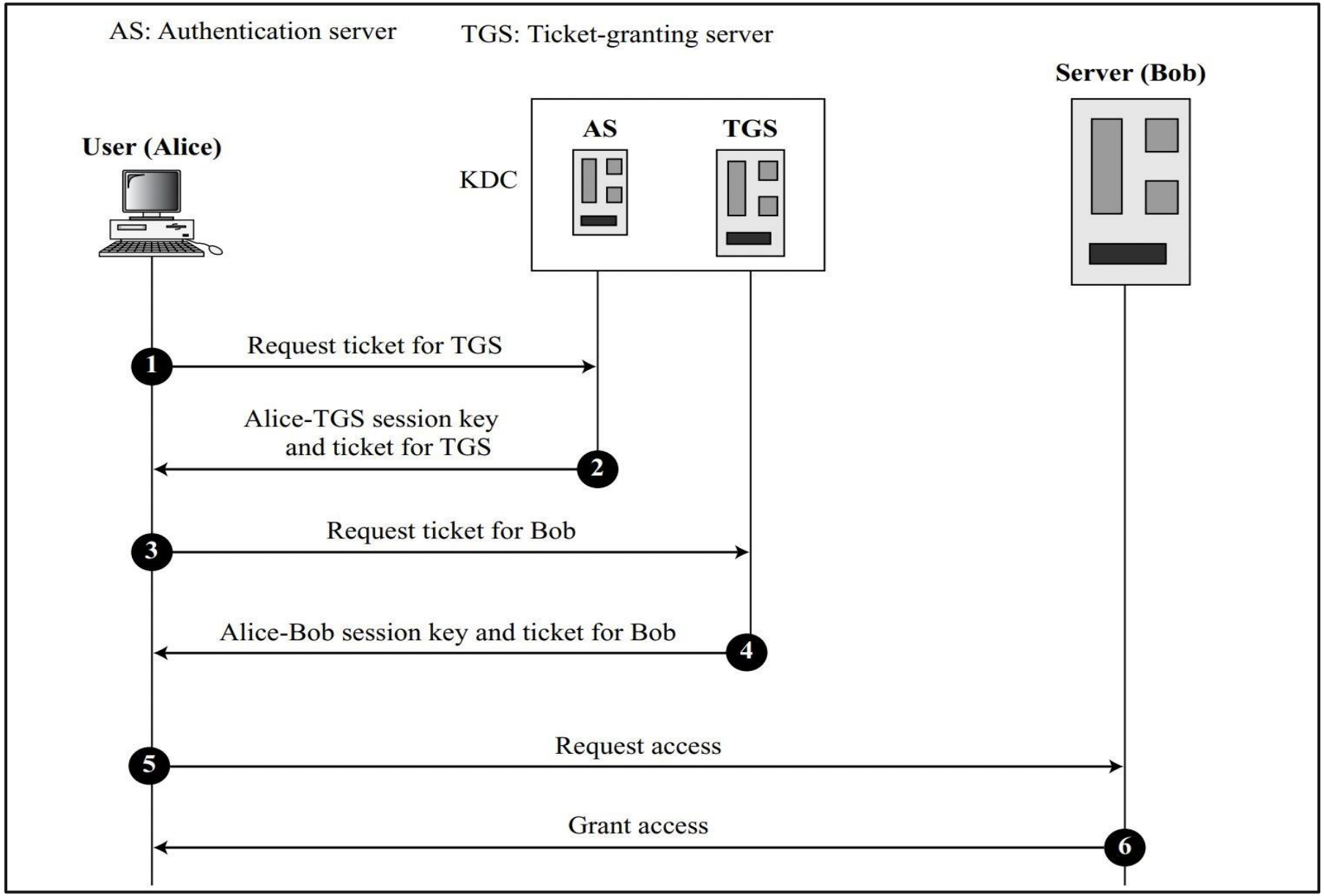
Authentication Protocol

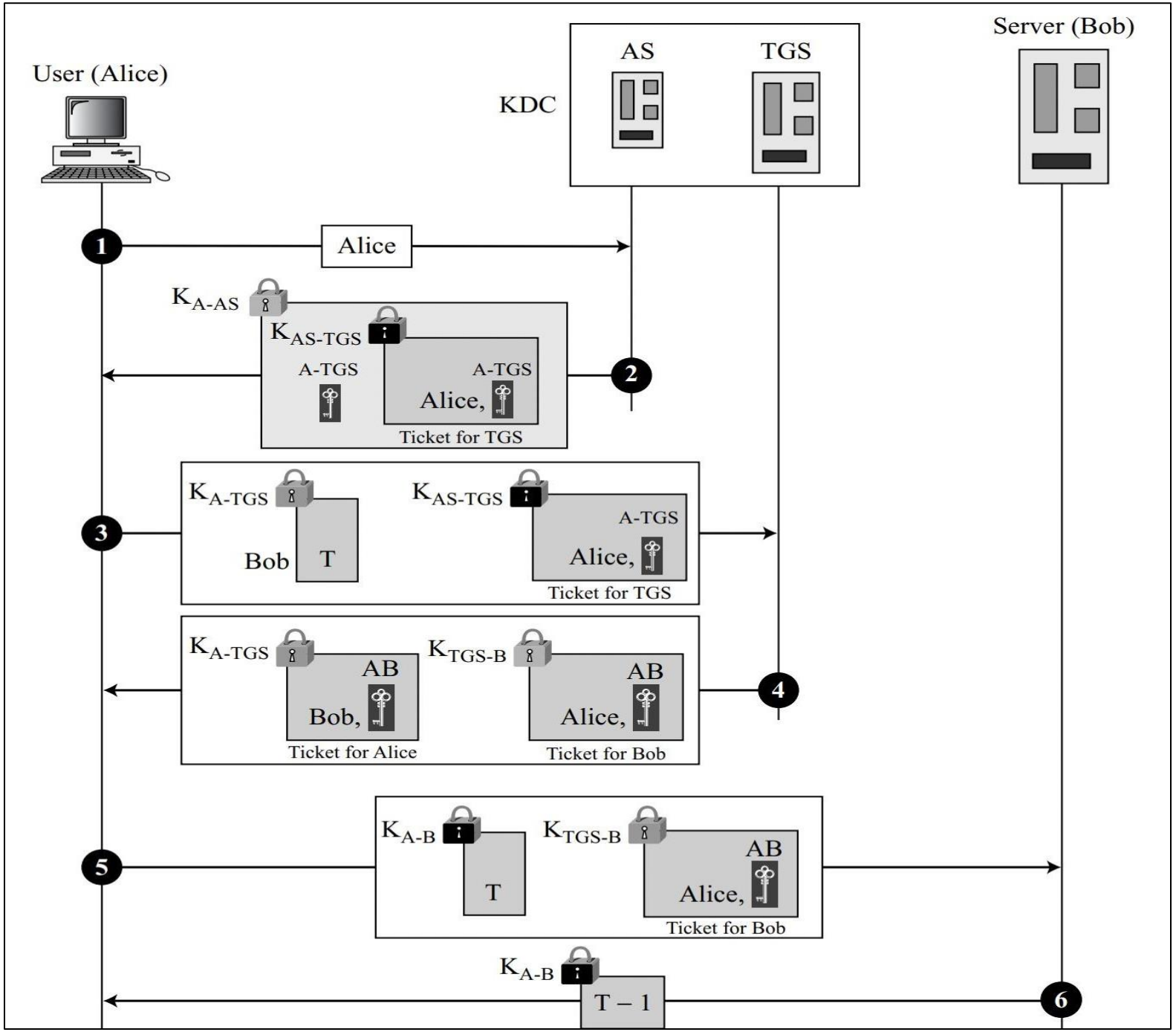
Kerberos - Authentication Protocol

Kerberos is a protocol for authenticating service requests between trusted hosts across an untrusted network, such as the internet. Kerberos support is built in to all major computer operating systems, including Microsoft Windows, Apple macOS, FreeBSD and Linux.

Kerberos - Authentication Protocol

Separate server for request user identification **(Authentication server)** and distributing session key**(Ticket Granting Server)**





K_{A-AS} Encrypted with Alice-AS key

K_{TGS-B} Encrypted with TGS-Bob key

K_{A-TGS} Encrypted with AS-TGS key

K_{AS-TGS} Encrypted with Alice-TGS session key

K_{A-B} Encrypted with Alice-Bob session key

A-TGS
Alice-TGS session key

AB
Alice-Bob session key

KDC: Key-distribution center
AS: Authentication server
TGS: Ticket-granting server
T: Timestamp (nonce)

Authentication Server (AS)

The **authentication server (AS)** is the KDC in the Kerberos protocol. Each user registers with the AS and is granted a user identity and a password. The AS has a database with these identities and the corresponding passwords. The AS verifies the user, issues a session key to be used between Alice and the TGS, and sends a ticket for the TGS.

Ticket-Granting Server (TGS)

The **ticket-granting server (TGS)** issues a ticket for the real server (Bob). It also provides the session key (K_{AB}) between Alice and Bob. Kerberos has separated user verification from the issuing of tickets. In this way, though Alice verifies her ID just once with the AS, she can contact the TGS multiple times to obtain tickets for different real servers.

Real Server

The real server (Bob) provides services for the user (Alice). Kerberos is designed for a client-server program, such as FTP, in which a user uses the client process to access the server process. Kerberos is not used for person-to-person authentication.

Operation

A client process (Alice) can access a process running on the real server (Bob) in six steps, as shown in Figure 15.8.

1. Alice sends her request to the AS in plain text using her registered identity.
2. The AS sends a message encrypted with Alice's permanent symmetric key, $K_{A\text{-}AS}$. The message contains two items: a session key, $K_{A\text{-}TGS}$, that is used by Alice to contact the TGS, and a ticket for the TGS that is encrypted with the TGS symmetric key, $K_{AS\text{-}TGS}$. Alice does not know $K_{A\text{-}AS}$, but when the message arrives, she types her symmetric password. The password and the appropriate algorithm together create $K_{A\text{-}AS}$ if the password is correct. The password is then immediately destroyed; it is not sent to the network and it does not stay in the terminal. It is used only for a moment to create $K_{A\text{-}AS}$. The process now uses $K_{A\text{-}AS}$ to decrypt the message sent. $K_{A\text{-}TGS}$ and the ticket are extracted.
3. Alice now sends three items to the TGS. The first is the ticket received from the AS. The second is the name of the real server (Bob), the third is a timestamp that is encrypted by $K_{A\text{-}TGS}$. The timestamp prevents a replay by Eve.
4. Now, the TGS sends two tickets, each containing the session key between Alice and Bob, $K_{A\text{-}B}$. The ticket for Alice is encrypted with $K_{A\text{-}TGS}$; the ticket for Bob is encrypted with Bob's key, $K_{TGS\text{-}B}$. Note that Eve cannot extract K_{AB} because Eve does not know $K_{A\text{-}TGS}$ or $K_{TGS\text{-}B}$. She cannot replay step 3 because she cannot replace the timestamp with a new one (she does not know $K_{A\text{-}TGS}$). Even if she is very quick and sends the step 3 message before the timestamp has expired, she still receives the same two tickets that she cannot decipher.
5. Alice sends Bob's ticket with the timestamp encrypted by $K_{A\text{-}B}$.
6. Bob confirms the receipt by adding 1 to the timestamp. The message is encrypted with $K_{A\text{-}B}$ and sent to Alice.

Kerberos Version 5

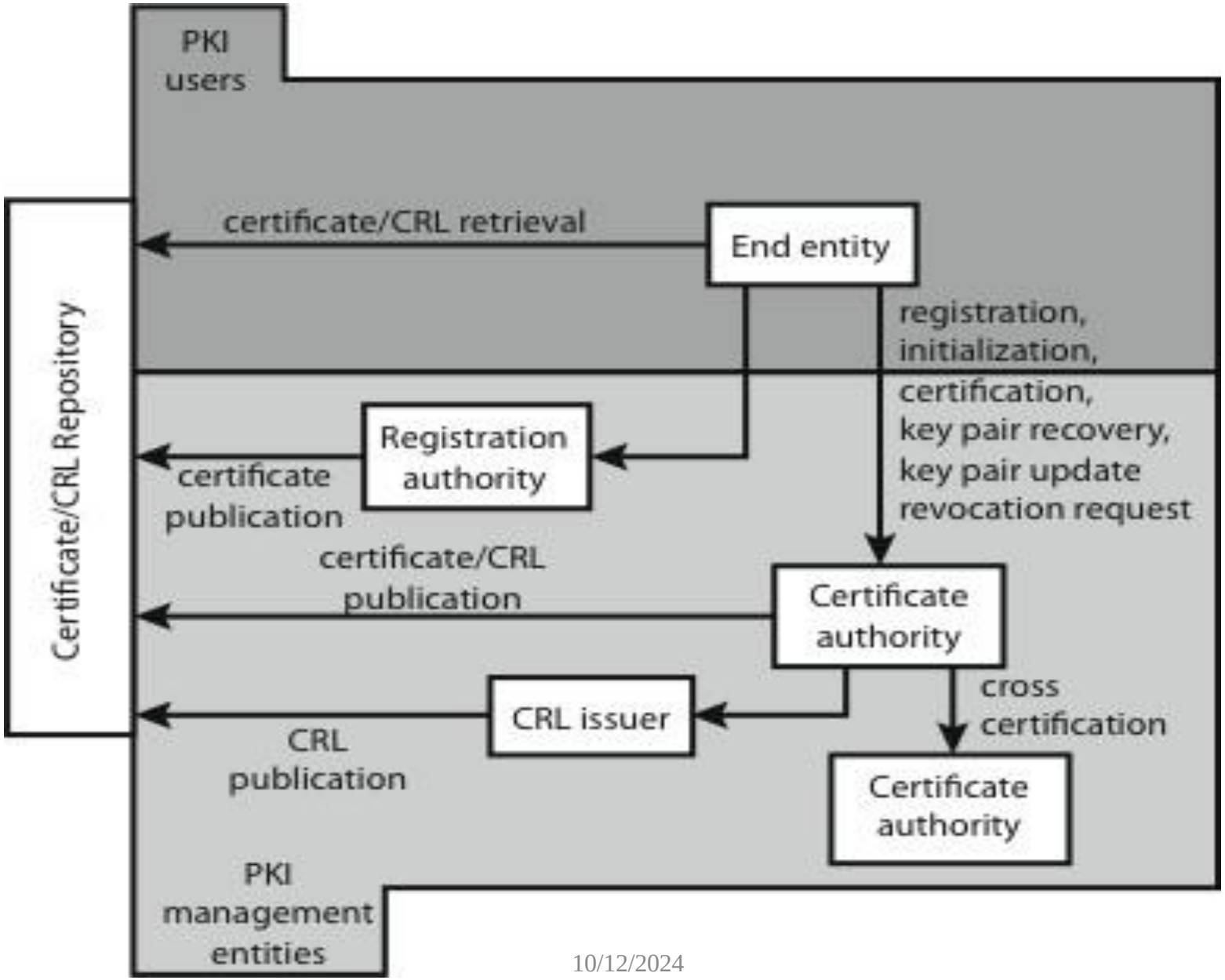
The minor differences between version 4 and version 5 are briefly listed below:

1. Version 5 has a longer ticket lifetime.
2. Version 5 allows tickets to be renewed.
3. Version 5 can accept any symmetric-key algorithm.
4. Version 5 uses a different protocol for describing data types.
5. Version 5 has more overhead than version 4.

Realms

Kerberos allows the global distribution of ASs and TGSs, with each system called a *realm*. A user may get a ticket for a local server or a remote server. In the second case, for example, Alice may ask her local TGS to issue a ticket that is accepted by a remote TGS. The local TGS can issue this ticket if the remote TGS is registered with the local one. Then Alice can use the remote TGS to access the remote real server.

Public Key Infrastructure



PKIX Management

- Functions
  - ✓ registration
  - ✓ initialization
  - ✓ certification
  - ✓ key pair recovery
  - ✓ key pair update
  - ✓ revocation request
  - ✓ cross certification
- Protocols
  - ✓ CMP
  - ✓ CMC